

1. Introduction & Scope

This document specifies the Functional Requirements (FRs) for the Machine Learning Recommendation System that recommends the optimal data centre cooling technique — Air Economizer, Evaporative Cooling, or Chilled Water — for a given operating scenario. The system uses a supervised Random Forest classifier trained on simulation-generated labelled data, combined with multi-criteria scoring and natural-language justification at inference time.

The scope of this document covers:

- Dataset generation pipeline (Monte Carlo sampling + three-simulator evaluation)
- Data quality and validation rules applied before training
- Model training, evaluation, and serialisation
- Recommendation API: ML prediction, feasibility override, and response structure
- Multi-criteria composite scoring (cost, emissions, water)
- Natural-language justification generation
- Multi-year future impact projection

Out of scope: LLM advisory engine, what-if analysis API, NSGA-II optimiser, Bayesian calibrator, Model Predictive Control, and climate risk assessor. Those components are documented separately.

1.1 Requirement ID Convention

Each requirement uses the format FR-XX-NN where XX is the area code and NN is a two-digit sequence number.

Prefix	Functional Area	Count
FR-DG	Dataset Generation	7
FR-DQ	Data Quality & Validation	6
FR-TR	Model Training & Evaluation	6
FR-RA	Recommendation API	6
FR-SC	Multi-Criteria Scoring	5
FR-JU	Justification Generation	3
FR-FI	Future Impact Projection	3
	TOTAL	36

Table 1 – Requirement area codes

2. Dataset Generation Requirements (FR-DG)

Source: Methodology §1–§7 + dataset.csv / dataset_full.csv. These requirements govern how the labelled training dataset is produced from the three cooling simulators using Monte Carlo scenario sampling.

FR-DG-01	Scenario Parameter Sampling
Description	The system SHALL generate operating scenarios by randomly sampling six input variables within defined operational bounds using uniform random sampling (Monte Carlo strategy) to ensure broad, unbiased coverage of the input space.
Inputs	Sampling bounds configuration: tempC [15–45 °C], rh [20–80 %], itLoadKW [100–2000 kW], electricityPrice [0.05–0.30 USD/kWh], waterPrice [0.50–3.00 USD/L], carbonFactor [0.10–0.90 kgCO ₂ /kWh].
Outputs	One scenario row containing all six input variables per sample.
Priority	HIGH
Source	Methodology §3–§4; dataset.csv column schema

FR-DG-02	Multi-Simulator Parallel Evaluation
Description	For each sampled scenario, the system SHALL submit identical input parameters to all three cooling simulators — Air Economizer, Evaporative Cooling, and Chilled Water — and collect outputs from each. The same inputs MUST be used for all three simulators to ensure a fair, unbiased cross-technique comparison.
Inputs	One scenario row (tempC, rh, itLoadKW, electricityPrice, waterPrice, carbonFactor).
Outputs	Per-technique outputs for each of the three simulators: feasible flag, energyKWh, waterLiters, cost, emissionsKg, violations count.
Priority	HIGH
Source	Methodology §5; dataset_full.csv columns: air_*, evap_*, chill_*

FR-DG-03	Feasibility-Gated Label Assignment
Description	The system SHALL assign the bestTechnique label using a strict two-step rule: (1) filter to only feasible techniques (feasible = True AND violations = 0); (2) select the technique with minimum energy consumption among the feasible set. If no technique is feasible, the row SHALL be labelled 'NoValidOption'.
Inputs	Per-technique feasibility flags and energyKWh values from all three simulators.
Outputs	bestTechnique label: one of {AirEconomizer, Evaporative, ChilledWater, NoValidOption}. Formal rule: bestTechnique _i = argmin _{t ∈ F_i} E _{i,t} where F _i is the feasible set.
Priority	HIGH
Source	Methodology §6; recommend_api.py _choose_best()

FR-DG-04	Compact Labelled Dataset (dataset.csv)
----------	--

Description	The system SHALL produce a compact labelled dataset containing only the six input features and the bestTechnique label. Per-technique outcome columns SHALL NOT be included in this file to prevent target leakage during classifier training.
Inputs	Full diagnostic dataset rows.
Outputs	CSV file with exactly 7 columns: tempC, rh, itLoadKW, electricityPrice, waterPrice, carbonFactor, bestTechnique.
Priority	HIGH
Source	Methodology §7; dataset.csv

FR-DG-05	Full Diagnostic Dataset (dataset_full.csv)
Description	The system SHALL produce a full diagnostic dataset containing all six input features, all per-technique outputs (feasible, energy, water, cost, emissions, violations for each of the three techniques), and the bestTechnique label. This file is used for analysis, explainability, and validation only — never as a training source.
Inputs	Simulator outputs for all three techniques per scenario.
Outputs	CSV file with 25 columns: 6 inputs + 6×3 per-technique outputs + bestTechnique.
Priority	MEDIUM
Source	Methodology §7; dataset_full.csv

FR-DG-06	Minimum Dataset Size
Description	The system SHALL generate a minimum of 500 labelled scenarios per dataset version. The recommended production size is 2 000+ rows to ensure sufficient class representation across all three technique labels.
Inputs	N parameter (number of scenarios to generate).
Outputs	Dataset CSV with $N \geq 500$ rows (excluding header row).
Priority	MEDIUM
Source	Methodology §4; dataset.csv (500 rows observed)

FR-DG-07	Reproducible Random Seed
Description	The system SHALL support a configurable random seed parameter for scenario generation so that datasets can be reproduced exactly for research reporting and regression testing. The default seed SHALL be 42.
Inputs	Seed value (integer).
Outputs	Deterministic, identical dataset output for the same seed value and N.
Priority	MEDIUM
Source	Methodology §11 assumption 4; train_recommender.py random_state=42

3. Data Quality & Validation Requirements (FR-DQ)

Source: Methodology §8 — Data Cleaning and Validation. These requirements ensure the dataset is clean and reproducible before model training.

FR-DQ-01	Required Column Enforcement	
Description	The system SHALL verify that all required columns are present in the dataset before training begins. If any required column is missing, the system SHALL raise a descriptive error identifying the missing column and halt processing immediately.	
Inputs	Dataset CSV file path.	
Outputs	Validation pass (all columns present) or descriptive KeyError identifying the missing column.	
Priority	HIGH	
Source	Methodology §8 step 1; train_recommender.py column check	

FR-DQ-02	Numeric Type Coercion	
Description	The system SHALL convert all six input feature columns to numeric (float64) type during data loading using pandas.to_numeric with errors='coerce'. Non-convertible string values SHALL be coerced to NaN rather than raising an error.	
Inputs	Raw CSV with potentially mixed-type columns.	
Outputs	DataFrame with float64 feature columns; non-numeric values converted to NaN.	
Priority	HIGH	
Source	Methodology §8 step 2	

FR-DQ-03	Missing Value Removal	
Description	The system SHALL drop any row that contains a missing value (NaN) in any of the six input feature columns or the bestTechnique label column before training. The count of dropped rows SHALL be logged.	
Inputs	DataFrame with potential NaN values.	
Outputs	Clean DataFrame with zero missing values in required columns; dropped row count logged.	
Priority	HIGH	
Source	Methodology §8 step 3	

FR-DQ-04	Duplicate Row Removal	
----------	-----------------------	--

Description	The system SHALL remove exact duplicate rows from the dataset before training to prevent the model from overfitting to repeated scenarios. The count of removed duplicates SHALL be logged.
Inputs	DataFrame with potential duplicate rows.
Outputs	DataFrame with unique rows only; duplicate count logged.
Priority	MEDIUM
Source	Methodology §8 step 4

FR-DQ-05	Label Normalisation and Validation
Description	The system SHALL normalise bestTechnique label strings (strip whitespace, apply consistent capitalisation) and validate that all labels belong to the allowed set: {AirEconomizer, Evaporative, ChilledWater}. Rows with invalid or unrecognised labels SHALL be excluded from training and their count reported.
Inputs	Raw bestTechnique column values.
Outputs	Normalised label column containing only allowed values; count of excluded invalid-label rows.
Priority	HIGH
Source	Methodology §8 step 5

FR-DQ-06	Class Distribution Reporting
Description	The system SHALL compute and log the class distribution — count and percentage per label — after all cleaning steps are complete. If any class represents less than 5% of the cleaned dataset, a class imbalance warning SHALL be raised to alert the user before training proceeds.
Inputs	Cleaned dataset with validated labels.
Outputs	Class distribution report (count + % per class); imbalance warning if any class < 5%.
Priority	MEDIUM
Source	Methodology §8 step 6

4. Model Training & Evaluation Requirements (FR-TR)

Source: Methodology §9 — Training Readiness Protocol; train_recommender.py. These requirements govern how the Random Forest classifier is trained, evaluated, and serialised as a reusable model artifact.

FR-TR-01	Feature Set Definition
Description	The system SHALL train the classifier using exactly the six input features defined in the dataset schema: tempC, rh, itLoadKW, electricityPrice, waterPrice, carbonFactor. No per-technique outcome columns (energy, cost, emissions, water, violations) SHALL be included as features, as this would constitute target leakage.
Inputs	Cleaned dataset.csv.
Outputs	Feature matrix X with shape (N, 6); target vector y with N class labels.
Priority	HIGH
Source	Methodology §7 guardrail; train_recommender.py features list

FR-TR-02	Stratified Train / Test Split
Description	The system SHALL split the cleaned dataset into a training set (80%) and a test set (20%) using stratified sampling by class label, ensuring that the proportion of each class is preserved in both splits. The random seed SHALL be fixed at 42.
Inputs	Cleaned feature matrix X and label vector y.
Outputs	X_train, X_test, y_train, y_test with stratified class distribution; split sizes logged.
Priority	HIGH
Source	Methodology §9 step 1; train_recommender.py train_test_split(stratify=y, random_state=42)

FR-TR-03	Random Forest Classifier Training
Description	The system SHALL train a Random Forest classifier with n_estimators=300 decision trees and random_state=42 for reproducibility. The classifier SHALL be fit exclusively on the training split — never on the test split.
Inputs	X_train, y_train.
Outputs	Trained RandomForestClassifier object ready for evaluation and serialisation.
Priority	HIGH
Source	train_recommender.py RandomForestClassifier(n_estimators=300, random_state=42)

FR-TR-04	Classification Report Generation
----------	----------------------------------

Description	The system SHALL evaluate the trained model on the held-out test split and produce a full classification report including per-class precision, recall, F1-score, support, and macro/weighted averages. The minimum acceptable macro F1-score is 0.80.
Inputs	Trained model, X_test, y_test.
Outputs	Printed classification report; pass/fail flag based on macro F1 $\geq$ 0.80 threshold.
Priority	HIGH
Source	Methodology §9 step 4; train_recommender.py classification_report

FR-TR-05	Model Artifact Serialisation
Description	The system SHALL serialise the trained model as a joblib artifact containing both the fitted model object and the feature_cols list (in training order). The artifact SHALL be saved to a configurable file path, defaulting to cooling_recommender_rf.pkl.
Inputs	Trained RandomForestClassifier object; feature column list.
Outputs	PKL file containing: {model: RandomForestClassifier, feature_cols: list[str]}.
Priority	HIGH
Source	train_recommender.py joblib.dump; recommend_api.py MODEL_PATH env var

FR-TR-06	Model Health Check Endpoint
Description	The recommendation API SHALL expose a GET /health endpoint that returns the model load status, the model file path, and the feature column list. If the model artifact failed to load at startup, the endpoint SHALL return status='error' with the error message so operators can diagnose the issue without inspecting logs.
Inputs	HTTP GET /health request.
Outputs	JSON response: {status: 'ok' 'error', model_loaded: bool, model_path: str, features: list[str]} or {status: 'error', error: str}.
Priority	HIGH
Source	recommend_api.py /health endpoint

5. Recommendation API Requirements (FR-RA)

Source: `recommend_api.py` — POST `/recommend` endpoint. These requirements define the core inference behaviour: how the API accepts a scenario, derives ML inputs, runs the classifier, applies feasibility logic, and returns a structured recommendation.

FR-RA-01	Simulation-Averaged Input Derivation
Description	The system SHALL derive the <code>tempC</code> , <code>rh</code> , and <code>itLoadKW</code> feature values for ML prediction by computing the arithmetic mean of the corresponding hourly arrays supplied in the <code>simulation_hourly</code> field of the request. Static user-provided scenario values for these three fields SHALL be overridden by the simulation averages.
Inputs	<code>simulation_hourly</code> dict containing arrays: <code>tempC[]</code> , <code>rh[]</code> , <code>itLoadKW[]</code> (<code>length ≥ 1</code>). <code>electricityPrice</code> , <code>waterPrice</code> , <code>carbonFactor</code> taken from scenario object as-is.
Outputs	Scalar averages <code>avg_tempC</code> , <code>avg_rh</code> , <code>avg_itLoadKW</code> used as ML feature inputs. Number of hours processed logged to console.
Priority	HIGH
Source	<code>recommend_api.py _mean_or_none()</code> ; <code>simulation_hourly</code> averaging block

FR-RA-02	simulation_hourly Mandatory Validation
Description	The system SHALL reject any POST <code>/recommend</code> request that does not include a <code>simulation_hourly</code> payload, or where the payload contains empty or non-numeric arrays for <code>tempC</code> , <code>rh</code> , or <code>itLoadKW</code> . A 422 Unprocessable Entity response SHALL be returned with a descriptive error message explaining the requirement.
Inputs	HTTP POST <code>/recommend</code> request body.
Outputs	HTTP 422 with detail message if <code>simulation_hourly</code> is absent, or if any of <code>tempC[]</code> , <code>rh[]</code> , <code>itLoadKW[]</code> is empty or contains no finite numeric values.
Priority	HIGH
Source	<code>recommend_api.py</code> <code>simulation_hourly</code> validation block

FR-RA-03	ML Model Prediction
Description	The system SHALL construct a single-row feature DataFrame from the six scenario inputs (using simulation-averaged values for <code>tempC</code> , <code>rh</code> , <code>itLoadKW</code>) and pass it to the loaded Random Forest model to obtain a predicted <code>bestTechnique</code> label. The column order MUST match the <code>feature_cols</code> list stored in the model artifact.
Inputs	Six-feature scenario dict; loaded model artifact with <code>feature_cols</code> .
Outputs	<code>model_recommendation</code> : string — one of {AirEconomizer, Evaporative, ChilledWater}.
Priority	HIGH
Source	<code>recommend_api.py</code> <code>model.predict(frame)[0]</code>

FR-RA-04	Feasibility Override Logic
----------	----------------------------

Description	When technique_results are provided, the system SHALL apply a three-tier decision rule to determine the final recommendation: (1) if the ML-predicted technique is feasible (feasible=True AND violations ≤ threshold), use it as the final recommendation (decision_source = 'ml_feasible'); (2) if the ML prediction is infeasible, fall back to the feasible technique with the lowest composite score (decision_source = 'fallback_scoring_due_to_infeasible_ml'); (3) if no technique is feasible, retain the ML prediction and flag the situation (decision_source = 'ml_no_feasible_options').
Inputs	ML prediction string; technique_results list with feasible flags and violations counts.
Outputs	final_choice dict (the winning technique row); decision_source string recorded in response.
Priority	HIGH
Source	recommend_api.py _is_row_feasible(), decision_source logic

FR-RA-05	Current Technique Switch / Keep Advisory
Description	When the request includes a current_technique field, the system SHALL compare it against the final recommendation and prepend context-aware advisory messages: if the current technique matches the recommendation, the system SHALL advise to continue and focus on optimisation; if it differs, the system SHALL advise switching and explain why.
Inputs	current_technique string (optional, nullable).
Outputs	Two to three advisory sentences prepended to why_this_is_recommended list.
Priority	MEDIUM
Source	recommend_api.py current_technique comparison block

FR-RA-06	UTC Timestamp on Every Response
Description	Every POST /recommend response SHALL include a generated_at_utc field containing the ISO 8601 UTC timestamp of when the recommendation was generated, enabling audit trails and response caching validation.
Inputs	System clock at request processing time.
Outputs	generated_at_utc: string in format YYYY-MM-DDTHH:MM:SS.ffffffZ.
Priority	LOW
Source	recommend_api.py datetime.utcnow().isoformat() + 'Z'

6. Multi-Criteria Scoring Requirements (FR-SC)

Source: `recommend_api.py` — `_normalize()`, `_enrich_rows()`, `_build_comparison_table()`. These requirements define how the three techniques are scored and ranked when simulation results are provided alongside the ML prediction.

FR-SC-01	Min-Max Normalisation of Objectives
Description	The system SHALL normalise cost, emissions_kg, and water_liters across all three techniques using min-max normalisation before computing composite scores. If all three techniques have identical values for an objective (range = 0), the normalised value SHALL be 0.0 for all three to avoid division by zero.
Inputs	Raw cost, emissions_kg, water_liters values for all three techniques.
Outputs	Three normalised arrays in [0.0, 1.0], one per objective.
Priority	HIGH
Source	<code>recommend_api.py _normalize()</code> — $(arr - min) / (max - min)$

FR-SC-02	Configurable Weighted Composite Score
Description	The system SHALL compute a composite score for each technique as a weighted sum of the three normalised objectives: $score = w_cost \times cost_n + w_emissions \times emissions_n + w_water \times water_n$. Weights SHALL be configurable via environment variables <code>WEIGHT_COST</code> , <code>WEIGHT_EMISSIONS</code> , <code>WEIGHT_WATER</code> with defaults 0.5, 0.3, 0.2 respectively. A lower score indicates a better technique.
Inputs	Normalised objective values; weight configuration from environment.
Outputs	Composite score (float) per technique. Lower = better.
Priority	HIGH
Source	<code>recommend_api.py</code> <code>WEIGHT_COST=0.5</code> , <code>WEIGHT_EMISSIONS=0.3</code> , <code>WEIGHT_WATER=0.2</code>

FR-SC-03	Annualised Metric Computation
Description	The system SHALL compute annualised versions of cost, emissions, and water usage for each technique based on the <code>metrics_unit</code> field in the request. If <code>metrics_unit='hourly'</code> , values SHALL be multiplied by $24 \times 365 = 8\,760$. If <code>metrics_unit='annual'</code> , values SHALL be used as-is. Invalid <code>metrics_unit</code> values SHALL default to 'annual'.
Inputs	Per-technique cost, emissions_kg, water_liters; <code>metrics_unit</code> string.
Outputs	<code>annual_cost</code> , <code>annual_emissions_kg</code> , <code>annual_water_liters</code> per technique.
Priority	HIGH
Source	<code>recommend_api.py _annualize()</code> function

FR-SC-04	Ranked Comparison Table in Response
----------	-------------------------------------

Description	The system SHALL return a comparison_table array in the response containing all three techniques sorted by composite score in ascending order (best first). Each entry SHALL include: tech, feasible, score, cost, emissions_kg, water_liters, violations, annual_cost, annual_emissions_kg, annual_water_liters.
Inputs	Enriched technique rows with computed scores.
Outputs	comparison_table: list of 3 dicts sorted by score ascending.
Priority	HIGH
Source	recommend_api.py _build_comparison_table()

FR-SC-05	Violation Threshold Enforcement
Description	The system SHALL treat a technique as infeasible if its violations count exceeds the MAX_ALLOWED_VIOLATIONS threshold. This threshold SHALL be configurable via the MAX_ALLOWED_VIOLATIONS environment variable with a default of 0 (zero tolerance). Techniques exceeding the threshold SHALL be excluded from the primary recommendation pool but still appear in the comparison table.
Inputs	violations count per technique; MAX_ALLOWED_VIOLATIONS environment variable.
Outputs	Boolean feasibility flag per technique after violation threshold check.
Priority	MEDIUM
Source	recommend_api.py _is_row_feasible(), MAX_ALLOWED_VIOLATIONS env var

7. Justification Generation Requirements (FR-JU)

Source: `recommend_api.py` — `_build_user_reasons()`. These requirements define the context-aware, human-readable explanation sentences that accompany every recommendation.

FR-JU-01	Context-Aware Justification List
Description	The system SHALL generate a <code>why_this_is_recommended</code> list of human-readable sentences for each recommendation. The list SHALL always include: (1) a statement naming the recommended technique; (2) a feasibility confirmation with violation count; (3) up to four condition-triggered contextual reasons based on scenario thresholds; (4) one quantified comparison sentence per alternative technique. The total list SHALL be capped at 8 items.
Inputs	Scenario dict, <code>final_choice</code> dict, alternatives list (sorted by score).
Outputs	<code>why_this_is_recommended</code> : list of up to 8 plain-English strings.
Priority	HIGH
Source	<code>recommend_api.py _build_user_reasons()</code>

FR-JU-02	Condition-Triggered Contextual Reasons
Description	The system SHALL append specific contextual reasons based on the following scenario thresholds — each triggered independently: (a) <code>carbonFactor</code> ≥ 0.5 kgCO ₂ /kWh → high-carbon grid reason; (b) <code>electricityPrice</code> ≥ 0.15 USD/kWh → high-tariff reason; (c) <code>rh</code> $\geq 70\%$ → high-humidity reason; (d) <code>itLoadKW</code> ≥ 1400 kW → high-load reason. Each triggered reason SHALL reference the recommended technique by name.
Inputs	Scenario dict values for <code>carbonFactor</code> , <code>electricityPrice</code> , <code>rh</code> , <code>itLoadKW</code> .
Outputs	0 to 4 additional contextual reason strings appended to the justification list.
Priority	MEDIUM
Source	<code>recommend_api.py _build_user_reasons()</code> threshold checks

FR-JU-03	Quantified Alternative Comparison Sentences
Description	For each non-recommended technique, the system SHALL generate one comparison sentence that quantifies the annual cost difference, annual emissions difference, and annual water difference relative to the recommended technique. Positive differences (recommended is cheaper/lower) SHALL be described as savings; negative differences SHALL be described as additional cost or usage.
Inputs	<code>annual_cost</code> , <code>annual_emissions_kg</code> , <code>annual_water_liters</code> for the recommended technique and each alternative.
Outputs	Two comparison sentences (one per alternative) formatted as: 'Compared with [alt], choosing [best] can save about \$X per year, reduce emissions by Y kg CO ₂ per year, and save Z liters of water per year.'
Priority	HIGH
Source	<code>recommend_api.py _build_user_reasons()</code> alternatives loop, <code>_money()</code> , <code>_num()</code> helpers

8. Future Impact Projection Requirements (FR-FI)

Source: `recommend_api.py` — `_build_future_impact_paragraph()`. These requirements define the multi-year projection output included in every recommendation response when technique results are available.

FR-FI-01	Year 1 / 3 / 5 Cost, Emissions, and Water Projection
Description	The system SHALL compute and return a <code>future_impact_paragraph</code> containing projected cumulative values for the recommended technique at Year 1, Year 3, and Year 5. Projections SHALL use a constant annual rate assumption: $\text{Year } N = N \times \text{annual_value}$.
Inputs	<code>annual_cost</code> , <code>annual_emissions_kg</code> , <code>annual_water_liters</code> for the recommended technique.
Outputs	<code>future_impact_paragraph</code> : single plain-English string containing Year 1, Year 3, and Year 5 projections for cost (USD), emissions (kg CO ₂), and water (liters).
Priority	HIGH
Source	<code>recommend_api.py _build_future_impact_paragraph()</code>

FR-FI-02	Projection Narrative Format
Description	The future impact paragraph SHALL be written in plain English suitable for non-technical stakeholders. It SHALL include: the technique name, Year 1 values, Year 3 cumulative values, Year 5 cumulative values, and a closing sentence referencing budget planning, sustainability targets, and cooling reliability. Currency values SHALL be formatted as \$X,XXX and numeric values as X,XXX.
Inputs	Computed Year 1/3/5 projection values.
Outputs	Single human-readable paragraph string with formatted monetary and numeric values.
Priority	MEDIUM
Source	<code>recommend_api.py _build_future_impact_paragraph()</code> , <code>_money()</code> , <code>_num()</code>

FR-FI-03	Graceful Degradation Without Technique Results
Description	If <code>technique_results</code> are not provided in the request, the system SHALL still return a valid <code>model_recommendation</code> . The <code>future_impact_paragraph</code> SHALL contain a placeholder message stating that detailed projections require technique comparison metrics, rather than raising an error or returning null.
Inputs	Absent or null <code>technique_results</code> field.
Outputs	<code>future_impact_paragraph</code> : 'Future impact details are available when technique comparison metrics are provided in the request.'
Priority	MEDIUM
Source	<code>recommend_api.py</code> response default <code>future_impact_paragraph</code>

9. Complete Requirements Register

FR ID	Title	Area	Priority
FR-DG-01	Scenario Parameter Sampling	Dataset Generation	HIGH
FR-DG-02	Multi-Simulator Parallel Evaluation	Dataset Generation	HIGH
FR-DG-03	Feasibility-Gated Label Assignment	Dataset Generation	HIGH
FR-DG-04	Compact Labelled Dataset (dataset.csv)	Dataset Generation	HIGH
FR-DG-05	Full Diagnostic Dataset (dataset_full.csv)	Dataset Generation	MEDIUM
FR-DG-06	Minimum Dataset Size	Dataset Generation	MEDIUM
FR-DG-07	Reproducible Random Seed	Dataset Generation	MEDIUM
FR-DQ-01	Required Column Enforcement	Data Quality	HIGH
FR-DQ-02	Numeric Type Coercion	Data Quality	HIGH
FR-DQ-03	Missing Value Removal	Data Quality	HIGH
FR-DQ-04	Duplicate Row Removal	Data Quality	MEDIUM
FR-DQ-05	Label Normalisation and Validation	Data Quality	HIGH
FR-DQ-06	Class Distribution Reporting	Data Quality	MEDIUM
FR-TR-01	Feature Set Definition	Model Training	HIGH
FR-TR-02	Stratified Train / Test Split	Model Training	HIGH
FR-TR-03	Random Forest Classifier Training	Model Training	HIGH
FR-TR-04	Classification Report Generation	Model Training	HIGH
FR-TR-05	Model Artifact Serialisation	Model Training	HIGH
FR-TR-06	Model Health Check Endpoint	Model Training	HIGH
FR-RA-01	Simulation-Averaged Input Derivation	Recommendation API	HIGH
FR-RA-02	simulation_hourly Mandatory Validation	Recommendation API	HIGH
FR-RA-03	ML Model Prediction	Recommendation API	HIGH
FR-RA-04	Feasibility Override Logic	Recommendation API	HIGH
FR-RA-05	Current Technique Switch / Keep Advisory	Recommendation API	MEDIUM
FR-RA-06	UTC Timestamp on Every Response	Recommendation API	LOW
FR-SC-01	Min-Max Normalisation of Objectives	Scoring	HIGH
FR-SC-02	Configurable Weighted Composite Score	Scoring	HIGH
FR-SC-03	Annualised Metric Computation	Scoring	HIGH

FR ID	Title	Area	Priority
FR-SC-04	Ranked Comparison Table in Response	Scoring	HIGH
FR-SC-05	Violation Threshold Enforcement	Scoring	MEDIUM
FR-JU-01	Context-Aware Justification List	Justification	HIGH
FR-JU-02	Condition-Triggered Contextual Reasons	Justification	MEDIUM
FR-JU-03	Quantified Alternative Comparison	Justification	HIGH
FR-FI-01	Year 1/3/5 Cost, Emissions, Water Projection	Future Impact	HIGH
FR-FI-02	Projection Narrative Format	Future Impact	MEDIUM
FR-FI-03	Graceful Degradation Without Results	Future Impact	MEDIUM

Table 2 – Complete functional requirements register (36 requirements)

Priority Summary

Priority	Count	% of Total
HIGH	25	69%
MEDIUM	10	28%
LOW	1	3%
TOTAL	36	100%

Table 3 – Requirements by priority

10. Data Schema Reference

10.1 dataset.csv — Compact Training Dataset (7 columns)

Column	Type	Range / Values	Role in ML
tempC	float64	15–45 °C	Input feature
rh	float64	20–80 %	Input feature
itLoadKW	float64	100–2 000 kW	Input feature
electricityPrice	float64	0.05–0.30 USD/kWh	Input feature
waterPrice	float64	0.50–3.00 USD/L	Input feature
carbonFactor	float64	0.10–0.90 kgCO ₂ /kWh	Input feature
bestTechnique	string	{AirEconomizer, Evaporative, ChilledWater}	Target label

Table 4 – dataset.csv schema

10.2 dataset_full.csv — Diagnostic Dataset (25 columns)

Column Group	Columns	Used in Training?
Input features (×6)	tempC, rh, itLoadKW, electricityPrice, waterPrice, carbonFactor	YES
Air Economizer outputs (×6)	air_feasible, air_energyKWh, air_waterLiters, air_cost, air_emissionsKg, air_violations	NO — diagnostic only
Evaporative outputs (×6)	evap_feasible, evap_energyKWh, evap_waterLiters, evap_cost, evap_emissionsKg, evap_violations	NO — diagnostic only
Chilled Water outputs (×6)	chill_feasible, chill_energyKWh, chill_waterLiters, chill_cost, chill_emissionsKg, chill_violations	NO — diagnostic only
Target label (×1)	bestTechnique	YES (label only)

Table 5 – dataset_full.csv schema

10.3 POST /recommend — Request Schema

Field	Type	Required	Description
scenario.tempC	float	Yes	Ambient dry-bulb temperature (°C)
scenario.rh	float	Yes	Relative humidity (%)
scenario.itLoadKW	float	Yes	IT load (kW)
scenario.electricityPrice	float	Yes	Electricity tariff (USD/kWh)
scenario.waterPrice	float	Yes	Water tariff (USD/L)
scenario.carbonFactor	float	Yes	Grid carbon intensity (kgCO ₂ /kWh)

Field	Type	Required	Description
simulation_hourly	dict	Yes	Hourly arrays: tempC[], rh[], itLoadKW[]
current_technique	string null	No	User's currently deployed cooling technique
technique_results	list null	No	Per-technique simulation outputs for scoring
metrics_unit	string	No	'annual' (default) or 'hourly'

Table 6 – POST /recommend request schema

10.4 POST /recommend — Response Schema

Field	Type	Always Present?	Description
model_recommendation	string	Yes	ML-predicted or feasibility-overridden best technique
generated_at_utc	string	Yes	ISO 8601 UTC timestamp of recommendation
why_this_is_recommended	list[str]	Yes	Up to 8 plain-English justification sentences
future_impact_paragraph	string	Yes	Year 1/3/5 projection or placeholder message
comparison_table	list[dict]	No	3 techniques ranked by composite score (if technique_results provided)
current_technique	string null	Yes	Echo of the user's current technique from request

Table 7 – POST /recommend response schema

11. Assumptions & Limitations

11.1 Explicit Methodological Assumptions

- The three cooling simulators adequately represent each technique under the configured constraints and parameter ranges.
- The sampled input ranges (tempC 15–45 °C, rh 20–80%, itLoadKW 100–2000 kW) represent a realistic operating envelope for the target deployment contexts.
- The feasibility logic is reliable and consistently applied across all three technique simulators.
- Energy-minimum feasible winner is a valid label policy for baseline technique recommendation.
- electricityPrice, waterPrice, and carbonFactor are exogenous inputs provided by the user — they are not predicted variables.
- The Random Forest classifier generalises to unseen scenarios within the sampled input bounds.
- The constant annual rate assumption in future projections is acceptable for Year 1–5 planning horizons.

11.2 Known Limitations

Limitation	Impact	Mitigation
Synthetic sampling may not match real temporal distributions	Model may underperform on real-world seasonal patterns	Supplement with real operational data when available
Label policy based on minimum energy only	May underweight cost/water/emissions trade-offs	Multi-criteria scoring in API compensates at inference time
Rare edge scenarios may be underrepresented	Poor generalisation at distribution boundaries	Increase N; use stratified scenario generation
Simulator bias propagates into labels	Misconfigured simulator produces systematically wrong labels	Validate simulators against known benchmarks before dataset generation
Year N projections assume constant annual rates	Does not account for electricity price escalation or carbon tax increases	Noted in projection paragraph; full escalation model planned

Table 8 – Known limitations and mitigations